

Horizontal Pod Autoscaling (HPA) in Kubernetes

Overview

Horizontal Pod Autoscaler (HPA) automatically scales the number of replica pods in a Kubernetes Deployment, ReplicaSet, or StatefulSet based on CPU utilization, memory utilization, or custom metrics.

HPA dynamically increases or decreases the number of pods depending on workload demand.

This helps:

- Improve application availability
 - Handle traffic spikes automatically
 - Optimize resource utilization
 - Reduce manual scaling operations
-

How HPA Works

1. Metrics Collection

HPA uses Metrics Server to collect resource metrics.

Supported metrics include:

- CPU utilization
 - Memory utilization
 - Custom metrics
 - External metrics
-

Metrics Sources

Metric Type	Source
CPU/Memory	Metrics Server
Custom Metrics	Prometheus Adapter
External Metrics	Datadog, CloudWatch, Prometheus

2. Scaling Decision

HPA continuously compares:

Current Resource Usage
vs
Configured Threshold

Scale-Up Scenario

If:

CPU Usage > Target CPU Threshold

Then:

Increase Replica Count

Scale-Down Scenario

If:

CPU Usage < Target CPU Threshold

Then:

Decrease Replica Count

3. Scaling Execution

Kubernetes automatically updates the replica count of the workload.

Example:

Replicas: 2 → 5

or

Replicas: 5 → 2

Enable HPA in Kubernetes Cluster

Before using HPA, Metrics Server must be installed.

Step 1: Check Metrics Server

Run:

```
kubectl get deployment metrics-server -n kube-system
```

Expected output:

NAME	READY	UP-TO-DATE	AVAILABLE
metrics-server	1/1	1	1

Step 2: Install Metrics Server (If Not Installed)

Run:

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

Step 3: Verify Metrics Server Pods

```
kubectl get pods -n kube-system | grep metrics-server
```

Expected output:

metrics-server-xxxxxxxx-xxxxx	1/1	Running
-------------------------------	-----	---------

Step 4: Verify Pod Metrics

Run:

```
kubectl top pods
```

Example output:

NAME	CPU(cores)	MEMORY(bytes)
nginx-deployment-xxxxxx-xxxxx	5m	20Mi

Important Requirement

HPA requires resource requests to be defined.

Without resource requests:

```
resources:  
  requests:
```

HPA cannot calculate utilization percentage correctly.

HPA Configuration Example

Step 1: Create a Sample Deployment

Create a file named:

deployment.yaml

Sample Deployment YAML

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: nginx-deployment  
  
spec:  
  replicas: 1  
  
  selector:  
    matchLabels:  
      app: nginx  
  
  template:  
    metadata:  
      labels:  
        app: nginx  
  
    spec:  
      containers:  
        - name: nginx
```

```
image: nginx

resources:
  requests:
    cpu: "100m"
    memory: "128Mi"

  limits:
    cpu: "200m"
    memory: "256Mi"
```

Step 2: Apply the Deployment

Run:

```
kubectl apply -f deployment.yaml
```

Step 3: Create Horizontal Pod Autoscaler

Run the following command:

```
kubectl autoscale deployment nginx-deployment --cpu-percent=50 --min=1 --max=5
```

Explanation of Parameters

Parameter	Description
--cpu-percent=50	Scale when CPU exceeds 50%
--min=1	Minimum replica count
--max=5	Maximum replica count

Step 4: Verify HPA

Run:

```
kubectl get hpa
```

Example output:

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS
REPLICAS AGE				
nginx-deployment	Deployment/nginx-deployment	10%/50%	1	5
1 5m				

Describe HPA

For detailed information:

```
kubectl describe hpa nginx-deployment
```

This shows:

- Current metrics
- Target metrics
- Replica count
- Scaling events
- Conditions

Generate Load to Test HPA

You can generate CPU load to trigger autoscaling.

Example Busybox Load Generator

Run:

```
kubectl run -i --tty load-generator --rm --image=busybox /bin/sh
```

Inside the pod:

```
while true; do wget -q -O- http://nginx-deployment; done
```

Monitor HPA During Load Test

Run:

```
kubectl get hpa -w
```

Example:

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS
REPLICAS AGE				
nginx-deployment 4 10m	Deployment/nginx-deployment	85%/50%	1	5

This indicates:

- CPU reached 85%
- HPA increased replicas to 4

Check Scaled Pods

Run:

```
kubectl get pods
```

You should observe additional pods being created.

YAML-Based HPA Configuration

Instead of CLI, HPA can also be created using YAML.

Example HPA YAML

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nginx-hpa

spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nginx-deployment

  minReplicas: 1
  maxReplicas: 5

  metrics:
  - type: Resource
```

```
resource:  
  name: cpu  
  
target:  
  type: Utilization  
  averageUtilization: 50
```

Apply HPA YAML

```
kubectl apply -f hpa.yaml
```

Memory-Based HPA Example

Example:

```
metrics:  
- type: Resource  
  
resource:  
  name: memory  
  
target:  
  type: Utilization  
  averageUtilization: 70
```

Multiple Metrics Example

```
metrics:  
- type: Resource  
  resource:  
    name: cpu  
    target:  
      type: Utilization  
      averageUtilization: 50  
  
- type: Resource  
  resource:  
    name: memory  
    target:
```



```
type: Utilization
averageUtilization: 70
```

HPA Scaling Formula

Kubernetes calculates desired replicas using:

```
genui{"math_block_widget_always_prefetch_v2":{"content":"Desired Replicas =
Current Replicas "}}
```

Example:

```
Current Replicas = 2
Current CPU Usage = 80%
Target CPU Usage = 40%
```

Calculation:

```
genui{"math_block_widget_always_prefetch_v2":{"content":"Desired Replicas = 2 =
4"}}
```

Kubernetes increases replicas from:

2 → 4

Important Notes

HPA Works Only with Resource Requests

Always define:

```
resources:
  requests:
```

Otherwise:

- Metrics become unavailable
 - Scaling may fail
-

HPA with Cluster Autoscaler

HPA scales pods.

Cluster Autoscaler scales worker nodes.

Example Workflow

1. HPA increases pods.
 2. Cluster lacks resources.
 3. Pods become Pending.
 4. Cluster Autoscaler adds new nodes.
 5. Pods schedule successfully.
-

HPA with VPA

Avoid using HPA and VPA together on CPU/memory metrics.

This causes scaling conflicts.

Recommended Combination

Component	Purpose
HPA	Scale pod count
VPA	Optimize resource requests
Cluster Autoscaler	Scale worker nodes

Useful Commands

Check HPA

```
kubectl get hpa
```

Describe HPA

```
kubectl describe hpa <hpa-name>
```

Delete HPA

```
kubectl delete hpa <hpa-name>
```

Watch Pods

```
kubectl get pods -w
```

Watch Deployment Scaling

```
kubectl get deployment -w
```

Check Metrics

```
kubectl top pods  
kubectl top nodes
```

Troubleshooting

Error: Metrics Not Available

Check Metrics Server:

```
kubectl get deployment metrics-server -n kube-system
```

Error: Missing CPU Request

Ensure deployment contains:

```
resources:  
  requests:  
    cpu:
```

HPA Not Scaling

Check:

- Metrics Server
- Resource Requests
- HPA Events
- Deployment Metrics

Run:

```
kubectl describe hpa
```

Summary

Horizontal Pod Autoscaler:

- Automatically scales pod replicas
- Uses CPU, memory, or custom metrics
- Improves availability and scalability
- Works together with Cluster Autoscaler for full Kubernetes autoscaling